

Manual *ms2*

A Molecular Simulation Tool for Thermodynamic Properties

Kaiserslautern, März 2016

Inhaltsverzeichnis

1	Introduction	1
1.1	Program Summary	1
1.2	Citation information	2
1.3	Support, Development & maintenance	2
2	Installation	3
2.1	Downloading the source Code	3
2.2	Compiling <i>ms2</i>	3
2.3	Options for Making <i>ms2</i>	3
3	Running <i>ms2</i>	4
3.1	Running <i>ms2</i> on Windows or Linux	4
3.2	Running <i>ms2</i> in serial or parallel mode	4
3.3	Flowchart	4
3.4	Input structure	4
3.4.1	par - file	5
3.4.2	pm - file	5
3.5	Initial Conditions	5
3.6	Output structure	5
3.7	Command line options	6
4	Keywords	7
4.1	Keywords listed by category	7
4.2	Keywords listed alphabetical	7
4.3	parsing rules	7
5	Methods	8
5.1	Basic simulation techniques	8
5.2	Molecular Dynamics	9
5.2.1	Implemented Thermostat and Barostat	9
5.2.2	Integrators	10
5.3	Monte Carlo	10
5.3.1	Moves	10
5.3.2	Resize	10
5.4	Simulation Setup	10
5.5	Ensembles	10
5.5.1	multiple Ensembles for multiple state points	11
5.5.2	Micro-Canonical Ensemble (NVE)	11
5.5.3	Canonical Ensemble (NVT)	11
5.5.4	Isobaric-Isothermal Ensemble (NpT)	11
5.5.5	Grand-Canonical Ensemble (μ VT)	11
5.6	Interaction types	11
5.6.1	Dispersive and Repulsive Interactions	11
5.6.2	Electrostatic Interactions	12
5.6.3	Intramolecular Interactions	15
5.6.4	Mixing rules	15

5.7	Cutoff & long range correction	15
5.7.1	Reaction Field	15
5.7.2	Ewald summation	15
5.8	Computing physical properties	15
5.8.1	Thermal properties	16
5.8.2	Caloric properties	16
5.8.3	second derivatives	18
5.8.4	Transport properties	19
5.8.5	configurational properties	21
5.8.6	Henry' s law constant	22
5.8.7	second virial coefficient	22
5.9	statistical Errors	22
5.10	special features	22
5.10.1	Vapour-liquid equilibrium	22
5.10.2	osmotic pressure	22
5.10.3	Hydrogen bonding	22
5.10.4	humid air Ensemble	22
6	Potential Models in the ms2-distribution	23
7	Computational Aspects	24
7.1	Sequential Version	24
7.2	Parallelization	24
7.3	Vectorization & Memory	25
7.4	Benchmarks/ Test case	25
7.5	Structure of program code	25
8	Tools	29
8.1	<i>ms2par</i>	29
8.2	<i>ms2potmod</i>	29
8.3	<i>ms2chart</i>	29
8.4	<i>ms2molecules</i>	29
8.5	<i>res2gepar</i>	30
9	Examples & Tutorials	31
10	Error Messages	32
	Symbolverzeichnis	33
	Abkürzungsverzeichnis	34

1 Introduction

1.1 Program Summary

Ms2 has already been successfully used in the Industrial Fluid Properties Simulation Challenge **fluidproperties** organized by mainly US companies. It is a molecular simulation program that aims the calculation of thermodynamic properties that are needed for applications in the chemical industry, without requiring expert knowledge by the user. It was developed for rigid molecular models with a focus on condensed phases, covering mixtures containing an arbitrary number of small molecular species.

The simulation program *ms2* is optimized for a fast execution on a broad range of computer architectures, spanning from single processor PCs over PC-clusters and vector computers to high end parallel machines. It is a standalone Fortran90 code that does not require any libraries or have any other software prerequisites. *ms2* is provided as a source code. It only needs to be compiled. The structure of *ms2* is modular and object-oriented, except at the very core of energy and force calculations, where some structural compromises were made to achieve a better performance.

ms2 is aimed at homogeneous bulk systems in equilibrium. For VLE calculations, the coexisting phases are simulated independently and subsequently. Thus, it is usually sufficient to sample molecular systems containing in the order of 10^3 molecules to obtain statistically reliable results **lyubartsev** Applying standard cut-off radii, the intermolecular interactions are evaluated roughly up to the maximal distance, which is given by the edge length of the cubic simulation volume. Therefore, spatial decomposition schemes for parallelization are not rewarding. Instead, Plimpton's method **plimpton** was implemented for parallel execution of molecular dynamics (MD), which scales reasonably well up to 16 or 32 processors, depending on the particular architecture. For Monte-Carlo (MC), a optimal parallelization scheme was introduced, taking advantage of the stochastic nature of such simulations. The computing resources are splitted into single runs each on one node, sampling the phase space independently. Both parallelization methods allow for short response times¹. A typical molecular simulation of a vapor liquid equilibrium state point takes roughly six hours on a current workstation. The main expenses for determining thermodynamic data in silico are installation and maintenance of computing equipment and/or computing time, which is presently below US\$ 0.35/CPU-hour **economist**

The simulation program *ms2* presented here was developed in a long-time cooperation of engineers and computer scientists. With the release of *ms2*, it is intended to facilitate the transfer of a state of the art and user-friendly molecular modeling and simulation package to academic institutions and the chemical industry. The release package consists of the simulation program *ms2* itself and auxiliary feature tools for setup and analysis, including a simple 3D visualization tool to monitor the molecular trajectories. Furthermore, accurate molecular models for more than 100 small molecules are provided.

In section 2 of this manual, the installation of *ms2* is described on different platforms. Section 3 introduces the implemented molecular simulation techniques. In sections 4 and 5, *ms2* and the implemented thermodynamic properties are outlined. Section 6 gives details on executing

¹Under "response time" we understand the real time difference between submission and termination of a simulation run.

ms2, while section 7 provides some examples to demonstrate the use of the simulation program. Section 8 presents runtime tests and section 9 describes the auxiliary feature tools. Section 10 of this manual comments on adaptations of the software. In section 11 the error messages are addressed.

1.2 Citation information

ms2 is a noncommercial software, which was published in Computer Physics Communications. The paper along with its Supplementary Material can be downloaded from the *ms2* home page (<http://www.ms-2.de>). It is free of charge for scientific research after registration. If you use *ms2* for generating scientific results that you want to publish you need to acknowledge the use of *ms2* by citing the following paper:

S. Deublein, B. Eckl, J. Stoll, S. Lishchuk, G. Guevara-Carrion, C.W. Glass, T. Merker, M. Bernreuther, H. Hasse, J. Vrabec

Computer Physics Communication **182** (2011) 2350

doi:10.1016/j.cpc.2011.04.026

The paper along with its Supplementary Material can be downloaded from the *ms2* home page.

1.3 Support, Development & maintenance

The newest version of the *ms2* molecular simulation package is available through the Internet from the *ms2* home page at:

<http://www.ms-2.de>

The source code of *ms2* can be downloaded as tarball. Other mirrors of *ms2* are not authorized for distribution of *ms2* and should therefore be avoided.

ms2 users are obliged to report bugs. Please use the following email address for reporting:

bugs@ms-2.de

You may try using the following email address for general contact:

contact@ms-2.de

2 Installation

2.1 Downloading the source Code

2.2 Compiling *ms2*

Compiling on Linux

Compiling on Windows

2.3 Options for Making *ms2*

3 Running *ms2*

ms2 was designed to be an easily applicable simulation program. Therefore, the input files are restricted to one file for the definition of simulation scenario and one file for each of the molecular species that are used in the simulation. The output files contain structured information on the simulation. All calculated thermodynamic properties are summarized in one output file, which is straight forwardly readable and self-explaining. For a more detailed evaluation of the simulation, the instantaneous and running averages of the most important thermodynamic properties are written to other files. In total, the current status of the simulation and many more details are written to six files and can be accessed during execution. The structure of the input files as well as the output files is shown in Figure 3.1.

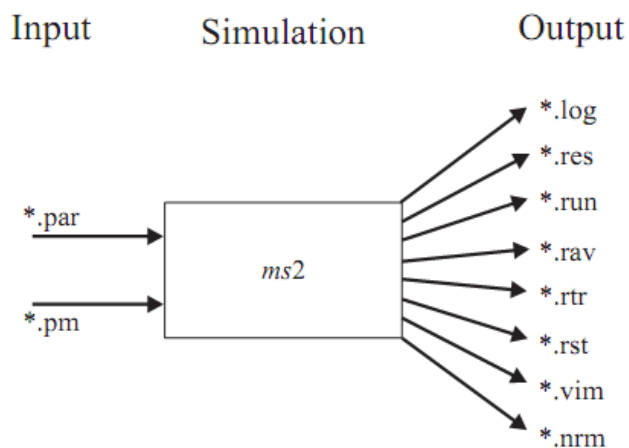


Abbildung 3.1: File structure needed and generated by *ms2*.

3.1 Running *ms2* on Windows or Linux

3.2 Running *ms2* in serial or parallel mode

3.3 Flowchart

3.4 Input structure

Each line contains one parameter string or is empty. In both cases the comment sign `#` and everything after it up to the end of the line is ignored. Parameter string consists of parameter name, `=` sign and parameter value. Leading spaces, trailing spaces, and spaces surrounding `=` sign are ignored. Parameter names and values for the parameter file and potential model files are described in sections 6.1.2 and 6.1.3 correspondingly. All parameters must be present in the input files in the certain order. The program searches for the parameters sequentially. Unidentified parameters are ignored. Missing or misplaced parameters lead to the termination of the program with the corresponding error message.

3.4.1 par - file

The simulation program *ms2* requires one input file (*.par) to specify the simulation parameters and one molecular model file (*.pm) for every molecular species considered. The *.par file contains all input variables for the simulation process, such as simulation type, ensemble, number of equilibration and production steps, time step length etc. Furthermore, the user has to specify temperature, density and fluid composition. Table 3 lists all input parameters and options to be specified in the *.par file. An example for a complete *.par file is given below for a MC simulation of pure ethylene oxide in the NpT ensemble, where the chemical potential is calculated by gradual insertion. Creating a *.par file is facilitated by the *ms2* feature program *ms2par*. Its use is described in detail in the section "Additional tools".

3.4.2 pm - file

A *.pm file contains the molecular model for a given substance. It contains the relative positions and the parameters of all sites. The potential model file for methanol is shown in Table ???. Methanol was modeled by two LJ sites and three point charges **schmabel**. All positions and distances in the *.pm file are given in Å, the LJ parameters σ and ϵ/k_B are given in Å and K, respectively, while the mass is given in atomic units ($u = 1.6605 \times 10^{-27}$ kg). The magnitudes of the charges are specified in electronic charges ($e = 1.602 \times 10^{-19}$ C), while the dipole moments and quadrupole moments are given in Debye ($D = 3.33564 \times 10^{-30}$ Cm) and Buckingham ($B = 3.33564 \times 10^{-40}$ Cm²), respectively. The orientations of the dipole and quadrupole are represented by spherical coordinates, where the azimuthal angle ϕ specifies the angle to the positive x-axis and the polar angle θ defines the angle to the positive z-axis. Both angles are specified in degrees. Molecular models can be orientated arbitrarily in the *.pm file. All site positions are transformed into a principal axes coordinate system at the beginning of each simulation with *ms2*. The normalized site positions are written to a *.nrm file for each component.

All input parameters and options to be specified in the *.pm file are given in table 5. A *.pm file has a specific structure, which is described in the following. The parameter *NSiteTypes* gives the number of different potential types (not the number of sites). Different potential types are 12-6 Lennard-Jones sites, point charges, point dipoles and point quadrupoles. Then the descriptions for each potential type follow. Each of them contains the number of sites of corresponding type followed by the description of each site. In the end the parameter *NRotAxes* is specified. Parameters *InertMomX*, *InertMomY* and *InertMomZ* are read if the moments of inertia are to be specified manually.

3.5 Initial Conditions

3.6 Output structure

ms2 yields eight output files:

- *.log file - stores a complete summary of all execution steps taken by *ms2*.
- *.res file - contains the results of the simulation in an aggregated form. The data is written to file in reduced quantities as well as in SI units, along with the statistical uncertainties of the calculated properties. The *.res file is created during simulation and updated every specified number of time steps or loops.

- *.run file - contains the calculated properties of the simulation for a specified time step or loop interval. The file is in tabular form, where the data is given in reduced units. The file is subsequently updated according to the user specification, which is set in the *.par file.
- *.rav file - contains the block averages of the calculated properties. The file is in tabular form, where the data is given in reduced units. The file is subsequently updated according to the user specification, which is set in the *.par file.
- *.rtr file - stores the final values of the autocorrelation functions and their integrals. The number of output lines has to be defined in the *.par file.
- *.rst file - is the restart file of the simulation. It contains all molecular positions, velocities, orientations, forces, torques and block averages for the thermodynamic properties. It is written once at the end of a simulation or immediately after having received a termination signal of the operating system. The *.rst file allows for a stepwise execution of the simulation, necessary e.g. in case of an early interruption of the simulation, time limits on a queuing system or unexpected halts.
- *.vim file - is the trajectory visualization file. It contains the positions and orientations of all molecules in an aggregated ASCII format. The configurations are written to file after a user-specified interval of time steps or loops. The *.vim file is readable by the visualization tool *ms2molecules*, which is also part of the simulation package.
- *.nrm file - stores the normalized coordinates of a potential model after a principal axes transformation.
- *.rtr file - stores the final value of the autocorrelation functions and their integrals. The number of output lines is equal to Corrlength divided by StepsCorrfun.

The next section provides some output files to illustrate the structure of them.

3.7 Command line options

4 Keywords

4.1 Keywords listed by category

4.2 Keywords listed alphabetical

4.3 parsing rules

5 Methods

5.1 Basic simulation techniques

The basic molecular simulation techniques employed in *ms2* are well described in the literature **allen**; **frenkel**; **rappaport** and are thus not repeated here. All non-standard methods that are implemented in *ms2* are introduced in section 5. The following paragraph briefly describes the most significant assumptions and techniques employed in *ms2*.

Simulations with *ms2* are performed in a quasi-infinite Cartesian space, using periodic boundary conditions **born** and the minimum image convention **metropolis**. The molecular interactions are considered to be pairwise additive, like in most other simulation packages. Including three- and many-body interactions leads to an increase in computational effort while the benefits are questionable. As usual, the computational effort in *ms2* is reduced by the introduction of a cut-off radius, up to which the intermolecular interactions are explicitly evaluated. The contributions of interactions with molecules beyond the cut-off radius are accounted for by correction schemes. For the electrostatic models, the reaction field method is included, while for the dispersive and repulsive interactions, a homogeneous fluid beyond the cut-off radius is assumed. The long range contributions are added to all thermodynamic data that are calculated in *ms2*.

Many thermodynamic properties calculated with *ms2* are residual quantities, indicated by the superscript "res". To compare them to data that are measured on the macroscopic level, the solely temperature dependent contributions of the ideal gas have to be added. These ideal properties are accessible e.g. by quantum chemical methods and can often be found in data bases **rowley**

The simulation program *ms2* internally uses reduced quantities for its calculations. All quantities are reduced by a reference length σ_R , a reference energy ϵ_R and a reference mass m_R , respectively. These reference values are input variables and may, in principle, be chosen arbitrarily. However, it is recommended to use a reference length σ_R in the order of 3 Å, a reference energy ϵ_R/k_B in the order of 100 K and a reference mass m_R in the order of 50 atomic units. The reduction scheme for the most important physical quantities is listed in table 5.1. From these properties, the reduced form of all other quantities can be derived. An exception in the reduction scheme is the chemical potential, which is normalized in *ms2* by $k_B T$ instead of ϵ_R .

Tabelle 5.1: Important physical quantities in their reduced form. Note that ϵ_0 indicates the permittivity of the vacuum $\epsilon_0 = 8.854187 \times 10^{-12} \text{A}^2 \text{s}^4 \text{kg}^{-1} \text{m}^{-3}$.

length	$l^* = \frac{l}{\sigma_R}$
energy	$u^* = \frac{u}{\epsilon_R}$
mass	$m^* = \frac{m}{m_R}$
time	$t^* = \frac{t}{\sigma_R} \sqrt{\frac{\epsilon_R}{m_R}}$
piston mass	$Q_P^* = \frac{Q_P \sigma_R^4}{m_R}$
temperature	$T^* = \frac{T k_B}{\epsilon_R}$
pressure	$p^* = \frac{p \sigma_R^3}{\epsilon_R}$
density	$\rho^* = \rho \sigma_R^3 N_A$
volume	$V^* = \frac{V}{\sigma_R^3}$
chemical potential	$\tilde{\mu} = \frac{\mu}{k_B T}$
point charge	$q^* = \frac{1}{\sqrt{4\pi\epsilon_0}} \frac{q}{\sqrt{\epsilon_R \sigma_R}}$
dipole moment	$\mu^* = \frac{1}{\sqrt{4\pi\epsilon_0}} \frac{\mu}{\sqrt{\epsilon_R \sigma_R^3}}$
quadrupole moment	$Q^* = \frac{1}{\sqrt{4\pi\epsilon_0}} \frac{Q}{\sqrt{\epsilon_R \sigma_R^5}}$
diffusion coefficient	$D^* = \frac{D}{\sigma \sqrt{m_R / \epsilon_R}}$
viscosity	$\eta^* = \frac{\eta \sigma_R^2}{\sqrt{m_R \epsilon_R}}$
thermal conductivity	$\lambda^* = \frac{\lambda \sigma_R^2}{k_B \sqrt{m_R / \epsilon_R}}$

5.2 Molecular Dynamics

Molecular dynamics (MD) meant to simulate the physical movements of particles (atoms, molecules, colloidal particles) of a given system. The path of the particles they follow through space as a function of time is determined from the forces acting between particles by numerically solving the Newton's equations of motion. These forces are described by atomistic force fields. Besides offering insight into molecular motion and providing direct structural information on an atomic scale MD allows to calculate macroscopic thermodynamic properties directly derived from the acting forces. In other words: we are able to obtain macroscopic properties from microscopic information. As MD follows the time evolution of the system, the outcome of a simulation is practically the time dependency of the calculated properties.

5.2.1 Implemented Thermostat and Barostat

In *ms2*, two integrators are implemented to solve Newton's equations of motion during MD simulation: Leapfrog and Gear predictor-corrector. These integrators are well known and described in literature **allen** The leapfrog integrator is a second order integrator that requires little computational effort, while being robust for many applications. The error of the method scales quadratically with the length of the chosen time step. The Gear predictor-corrector integrator is implemented with fifth order and is more accurate for small time steps compared to the Leapfrog integration **allen** The computational demand for both integration schemes is similar as implemented in *ms2*. The Gear integration, though being of higher order, is only 0.3% slower than the Leapfrog

algorithm for the total calculation of one MD time step with a molecular model composed of three LJ sites, having three rotational degrees of freedom.

5.2.2 Integrators

5.3 Monte Carlo

The Monte Carlo (MC) method is the application of statistical mechanics to describe molecular systems. Problems that can be simulated with MC can also be modeled by the MD method. The difference is that the MC approach relies on statistics rather than trying to reproduce the trajectory of particles in real time. For MD the solution of Newton's equations of motion is the trajectory of every particle in the system, the trajectories can be interpreted as a sequence of states (configurations). The main idea behind MC is to sample the set of possible configurations by selecting the ones that have significant contribution to the calculated properties. In the standard MC simulation procedure configurations of a given ensemble is generated in a stochastic way by performing trial moves of particles and accepting them with an appropriate transition probability.

5.3.1 Moves

For MC, the Markov chain is generated by repeatedly disturbing the system by translational or rotational motion of a molecule and evaluating the resulting configurations with respect to energy using Metropolis acceptance criterion. The thermostat incorporated in *ms2* is velocity scaling. Here, the velocities are scaled such that the actual kinetic energy matches the specified temperature. The scaling is applied equally over all molecular degrees of freedom. The pressure is kept constant using Andersen's barostat in MD, and random volume changes evaluated according to Metropolis acceptance criterion in MC, respectively.

5.3.2 Resize

5.4 Simulation Setup

5.5 Ensembles

Following ensembles are currently supported by *ms2*:

- canonical ensemble (NVT) - MD and MC
- micro-canonical ensemble (NVE) - MD
- isobaric-isothermal ensemble (NpT) - MD and MC
- grand equilibrium method (pseudo- μVT) - MC

Most of these ensembles are well known and widely in use **allen**. Therefore, the discussion is restricted here to the grand equilibrium method, since it was developed recently **vrabec1** and is probably new to many readers.

The grand equilibrium method is a technique to determine the VLE of pure substances or mixtures. It is a two-step procedure, where the coexisting phases are simulated independently and subsequently. The specified thermodynamic variables for the VLE are the temperature T and the composition $\mathbf{x}$ of the liquid. In the first step, one NpT simulation of the liquid phase is

performed at T , $\mathbf{x}$ and some pressure p_0 to determine the chemical potentials μ_i^l and the partial molar volumes v_i^l of all components, which corresponds to the molar volume in case of a pure substance. If the entropic properties are determined by Widom’s test molecule method **widom** both MD or MC can be used to sample the phase space. A more advanced technique for these properties is implemented in combination with MC, i.e. gradual insertion **lyubartsev1**; **nezbeda** (see below). On the basis of the chemical potentials and partial molar volumes at p_0 , first order Taylor expansions can be made for the pressure dependence

$$\mu_i^l(T, \mathbf{x}, p) \approx \mu_i^l(T, \mathbf{x}, p_0) + v_i^l(T, \mathbf{x}, p_0) \cdot (p - p_0) . \quad (5.1)$$

Note that *ms2* yields $\mu_i(T, \mathbf{x}, p) - \mu_i^{id}(T)$, where $\mu_i^{id}(T)$ is the temperature dependent part of the ideal gas contribution to the chemical potential of the pure component. $\mu_i^{l,id}(T)$ does not need to be determined for VLE calculations, because it cancels out when Eq. (5.1) is equated to the corresponding expression for the vapor.

In the second step, one pseudo- μVT simulation **vrabec1** is performed for the vapor phase on the basis of Eq. (5.1) that yields the saturated vapor state point of the VLE. This simulation takes place in a pseudo ensemble in the sense that the specified chemical potentials are not constant, but dependent on the actual pressure of the vapor phase. However, experience based on hundreds of systems **huang**; **vrabec** shows that the vapor phase simulation rapidly converges to the saturated vapor state point during equilibration so that effectively the equilibrium chemical potentials are specified via the attained vapor pressure. The number of molecules varies during the vapor phase simulation of the grand equilibrium method. Starting from a specified number of molecules in an arbitrarily chosen gaseous state, *ms2* adjusts the extensive volume V after an equilibration period so that the vapor density does not have to be known in advance.

The grand equilibrium method has been widely used and compares favorably to other methods used for vapor liquid equilibrium simulations.

5.5.1 multiple Ensembles for multiple state points

5.5.2 Micro-Canonical Ensemble (NVE)

5.5.3 Canonical Ensemble (NVT)

5.5.4 Isobaric-Isothermal Ensemble (NpT)

5.5.5 Grand-Canonical Ensemble (μVT)

5.6 Interaction types

5.6.1 Dispersive and Repulsive Interactions

In *ms2*, the dispersive and repulsive interactions between molecules are reduced to pairwise interactions u_{ij}^{LJ} of the different molecule sites i and j , which are modeled by the 12-6-LJ potential **lennard**

$$u_{ij}^{LJ} = 4\epsilon \left(\left(\frac{\sigma}{r_{ij}} \right)^{12} - \left(\frac{\sigma}{r_{ij}} \right)^6 \right) . \quad (5.2)$$

Here, the site-site distance between two interacting LJ sites i and j is denoted by r_{ij} , while σ and ϵ are the LJ size and energy parameters, respectively. The LJ potential is widely used and allows

for a fast computation of these basic interactions. It has only two parameters, which facilitates the parameterization of molecular models.

For pure components, the interactions between two different LJ sites are described by the Lorentz-Berthelot combination rules **berthelot**; **lorentz** For mixtures, the combination rules are extended to the modified Lorentz-Berthelot rules, which include two additional parameters η and ξ to describe the interactions between LJ sites of unlike molecules **fischer**

$$\sigma_{ij} = \eta \frac{\sigma_i + \sigma_j}{2} , \quad (5.3)$$

$$\epsilon_{ij} = \xi \sqrt{\epsilon_i \epsilon_j} . \quad (5.4)$$

The two parameters scale all LJ interactions between molecules of different components equally. Arbitrary modifications of the combination rules are possible. *ms2* allows the specification of η and ξ for every molecule pair independently and thus the free parameterization of the combination rules.

5.6.2 Electrostatic Interactions

ms2 considers electrostatic interactions between electro-neutral molecules. Three different electrostatic site type models are available: point charge, point dipole and linear point quadrupole. The two higher order electrostatic interaction sites integrate characteristic arrangements of several single partial charges on a molecule. The simulative advantage of higher order polarities is faster execution and a better description of the electrostatic potential of the molecule for a given number of electrostatic sites **stone** Combining two partial charges to one point dipole reduces the computational effort with *ms2* by about 30%, combining three point charges to one point quadrupole reduces the computational demand by even 60%. An additional advantage is the simplification of the molecular model, which reduces the number of molecular model parameters. In the following, the implemented electrostatic interactions between sites of the same type and between unlike site types are briefly described.

Point charge. Point charges are first order electrostatic interaction sites. The electrostatic interaction between two point charges q_i and q_j is given by Coulomb's law **allen**

$$u_{ij}^{qq}(r_{ij}, q_i, q_j) = \frac{1}{4\pi\epsilon_0} \frac{q_i q_j}{r_{ij}} . \quad (5.5)$$

This interaction decays with the inverse of r_{ij} and is therefore significant up to very large distances. For computational efficiency, the Coulombic contributions to the potential energy are explicitly evaluated up to a specified cut-off radius r_c . The long range interactions with charges beyond r_c are corrected for by the reaction field method. Its application to point charges is discussed below.

Point dipole. A point dipole describes the electrostatic field of two point charges with equal magnitude, but opposite sign at a mutual distance $a \rightarrow 0$. Its moment μ is defined by $\mu = qa$. The electrostatic interaction between two point dipoles with the moments μ_i and μ_j at a distance r_{ij} is given by **allen**; **gray**

$$u_{ij}^{DD}(r_{ij}, \theta_i, \theta_j, \phi_{ij}, \mu_i, \mu_j) = \frac{1}{4\pi\epsilon_0} \frac{\mu_i \mu_j}{r_{ij}^3} (\sin \theta_i \sin \theta_j \cos \phi_{ij} - 2 \cos \theta_i \cos \theta_j) , \quad (5.6)$$

with θ_i being the angle between the dipole direction and the distance vector of the two interacting dipoles and ϕ_{ij} being the azimuthal angle of the two dipole directions, cf. Figure 5.1. In *ms2*, the interaction between two dipoles is explicitly evaluated up to the specified cut-off radius r_c . The long range contributions are considered in *ms2* by the reaction field method **allen**

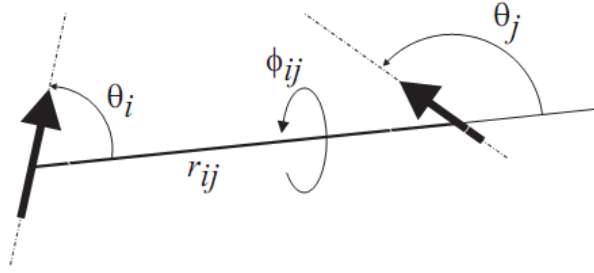


Abbildung 5.1: Schematic of the angles between two point dipoles i and j indicated by the arrows, which are situated on different molecules at a distance r_{ij} .

Linear point quadrupole. A linear point quadrupole describes the electrostatic field induced either by two collinear point dipoles with the same moment, but opposite orientation at a distance $a \rightarrow 0$, or by at least three collinear point charges with alternating sign $(q, -2q, q)$. The resulting quadrupole moment Q is defined by $Q = 2aq$. The electrostatic interaction between two linear point quadrupoles with the moments Q_i and Q_j at a distance r_{ij} is given by **gray; hirschfelder**

$$u_{ij}^{QQ}(r_{ij}, \theta_i, \theta_j, \phi_{ij}, Q_i, Q_j) = \frac{1}{4\pi\epsilon_0} \frac{3}{4} \frac{Q_i Q_j}{r_{ij}^5} [1 - 5((\cos \theta_i)^2 + (\cos \theta_j)^2) - 15(\cos \theta_i)^2 (\cos \theta_j)^2 + 2(\sin \theta_i \sin \theta_j \cos \phi_{ij} - 4 \cos \theta_i \cos \theta_j)^2], \quad (5.7)$$

where the angles θ_i , θ_j and ϕ_{ij} indicate the relative angular orientation of the two point quadrupoles, as discussed above. Note that no long range correction is necessary for this interaction type if the fluid is isotropic.

Point dipole interactions. The interaction between a point dipole with moment μ_i and a point charge q_j at a distance r_{ij} is given by **hirschfelder; lorrain**

$$u_{ij}^{Dq}(r_{ij}, \theta_i, \mu_i, q_j) = -\frac{1}{4\pi\epsilon_0} \frac{\mu_i q_j}{r_{ij}^2} \cos \theta_i. \quad (5.8)$$

Here, θ_i is the angle between the distance vector of the point charge and the orientation vector of the point dipole, as illustrated in Figure 5.1.

Point quadrupole interactions. The interaction potential of a linear point quadrupole Q_i with a point charge q_j at a distance r_{ij} is given by **allen; gray**

$$u_{ij}^{Qq}(r_{ij}, \theta_i, Q_i, q_j) = \frac{1}{4\pi\epsilon_0} \frac{Q_i q_j}{4r_{ij}^3} (3 \cos^2 \theta_i - 1), \quad (5.9)$$

where θ_i is the angle between the distance vector of the point charge and the orientation vector of the point quadrupole, as illustrated in Figure 5.1.

The interaction between a linear point quadrupole with moment Q_i and a point dipole with moment μ_j is given by **gray; hirschfelder**

$$u_{ij}^{Q\mu}(r_{ij}, \theta_i, \theta_j, \phi_{ij}, Q_i, \mu_j) = \frac{1}{4\pi\epsilon_0} \frac{3}{2} \frac{Q_i \mu_j}{r_{ij}^4} (\cos \theta_i - \cos \theta_j) (1 + 3 \cos \theta_i \cos \theta_j - 2 \cos \phi_{ij} \sin \theta_i \sin \theta_j), \quad (5.10)$$

where the angles θ_i , θ_j and ϕ_{ij} indicate the relative angular orientation of the point dipole j and the point quadrupole i , as shown in Figure 5.1.

Reaction field method. The truncation of interactions between first and second order electrostatic sites leads to errors that need to be corrected. The reaction field method **frenkel**; **nymand** is implemented in *ms2* for this task, being widely used and well accepted **steinhauser**; **vanderSpoel** Its advantages are accuracy and stability, while requiring little computational effort compared to other techniques like Ewald summation **ewald** However, it is limited to electro-neutral systems, thus *ms2* is currently restricted to electro-neutral molecules. The basic assumption of the reaction field method implemented in *ms2* is that the system is sufficiently large so that tinfoil boundary conditions ($\epsilon_s \rightarrow \infty$) are applicable without a loss of accuracy. This is the case for $N \geq 500$ **lisal**; **garzon**; **lisal1**; **jedlovsky**

All dipoles within the cut-off radius r_c polarize the fluid surrounding the cut-off sphere, which is modeled as a dielectric continuum with a relative permittivity ϵ_s . The polarization gives rise to a homogeneous electric field within the cut-off radius, called the reaction field $\mathbf{E}_i^{\text{RF}}$ of magnitude

$$\mathbf{E}_i^{\text{RF}} = \frac{1}{4\pi\epsilon_0} \frac{2(\epsilon_s - 1)}{2\epsilon_s + 1} \frac{1}{r_c^3} \sum_{b=1}^m \sum_{\substack{j=1 \\ r_{ij} < r_c}}^{N_b} \boldsymbol{\mu}_{b,j} . \quad (5.11)$$

Note that all m dipoles with moment $\boldsymbol{\mu}$ on all N_b molecules within the cut-off sphere have to be summed up. The reaction field acting outside of the cut-off sphere interacts with a dipole $\boldsymbol{\mu}_i$ at the center of the cut-off sphere by **allen**; **boettcher**

$$u_i^{\mu\text{RF}} = -\frac{1}{2} \boldsymbol{\mu}_i \mathbf{E}_i^{\text{RF}} , \quad (5.12)$$

where $u_i^{\mu\text{RF}}$ is its contribution to the potential energy. Eq. (5.12) makes use of the assumption that the system is sufficiently large.

The reaction field method can also be applied to correct for sets of point charges, as long as they add up to a total charge of zero. Therefore, a point charge distribution is reduced to a dipole vector $\boldsymbol{\mu}^q$ according to

$$\boldsymbol{\mu}^q = \sum_{i=1}^n \mathbf{r}_i q_i , \quad (5.13)$$

where n denotes the total number of point charges and $\mathbf{r}_i$ the position vector of charge q_i . The resulting dipole $\boldsymbol{\mu}^q$ is transferred into the correction term according to Eqs. (5.11) and (5.12).

5.6.3 Intramolecular Interactions

5.6.4 Mixing rules

5.7 Cutoff & long range correction

5.7.1 Reaction Field

5.7.2 Ewald summation

Ewald summation **allen1987**; **frenkel2003** was implemented for the calculation of electrostatic interactions between point charges. It extends the applicability of *ms2* to thermodynamic properties of e.g. ions in solutions. In Ewald summation, the electrostatic interactions according to Coulomb's law are divided into two contributions: short-range and long-range. The short-range term includes all charge-charge interactions at distances smaller than the cut-off radius. The remaining contribution is calculated in Fourier space and only the final value is transformed back into real space. This allows for an efficient calculation of the long-range interactions between the charges. The algorithm is well described in literature. Currently, some of the new features, the calculation of Massieu potential derivatives and Hybrid MPI & OpenMP Parallelization for MD, are not available together with Ewald summation.

5.8 Computing physical properties

The simulation program *ms2* calculates the thermodynamic properties from the trajectories (MD) or Markov Chains (MC) on the fly. The results are written to file with a specified frequency during the course of the simulation. The statistical uncertainties of all results are estimated using the block averaging method according to Flyvbjerg and Petersen **flyvberg** and the error propagation law.

The simulation program *ms2* allows for the determination of static and dynamic thermodynamic properties in equilibrium. The implemented static properties are:

- Thermal and caloric properties
- Chemical potential
- Vapor-liquid equilibria
- Henry's law constant
- Second virial coefficient

The transport properties can be calculated on the fly during a MD simulation with a reasonable additional computational effort using the Green-Kubo formalism **green**; **kubo** The implemented dynamic properties are:

- Self-diffusion coefficient
- Maxwell-Stefan diffusion coefficient
- Shear viscosity
- Bulk viscosity

5.8.1 Thermal properties

The static thermodynamic properties accessible via *ms2* depend on the chosen ensemble. At constant temperature and volume (NVT ensemble), e.g. the pressure is determined by the virial expression W . Other important thermodynamic properties are accessible at constant temperature and pressure. In the NpT ensemble, the volume is a fluctuating parameter that on average yields the volume corresponding to the specified temperature and pressure. In both ensembles, the residual internal energy is the sum of all pairwise interaction energies u_{ij} and the appropriate long range correction. The residual enthalpy is linked to the residual internal energy, pressure and volume of the system using the thermodynamic definition.

At constant temperature T and volume V , the pressure p is determined by **allen**

$$p = \frac{k_B T}{V} + \frac{\langle W \rangle}{V} + \Delta p^L = \frac{k_B T}{V} + \frac{1}{3V} \left\langle \sum_{i=1}^{N-1} \sum_{\substack{j=i+1 \\ r_{ij} < r_c}}^N \mathbf{r}_{ij} \mathbf{f}_{ij} \right\rangle + \Delta p^L, \quad (5.14)$$

where k_B is the Boltzmann constant and the brackets $\langle \dots \rangle$ indicate the ensemble average. W denotes the virial, which is defined by the force vector $\mathbf{f}_{ij}$ acting between two molecules at a separation vector $\mathbf{r}_{ij}$ between their centers of mass. The contribution Δp^L considers the long-range interactions with molecules beyond the cut-off radius r_c .

In the isothermal-isobaric ensemble, the volume is a fluctuating parameter that on average yields the volume corresponding to the specified pair of temperature T and pressure p_0 . In MD simulations with Andersen's barostat **andersen** which is implemented in *ms2*, the fluctuations of the volume are damped by a fictive piston mass Q_p according to the equation of motion

$$\ddot{V} = \frac{p - p_0}{Q_p}. \quad (5.15)$$

In MC simulations, the volume is changed randomly and the new volume accepted by applying the Metropolis acceptance criterion

$$P_{acc} = \min \left(1, \exp \left(\frac{\Delta E}{k_B T} \right) \right), \quad (5.16)$$

where P_{acc} is the probability of accepting the volume change and ΔE is the difference between the old state with energy U_{old} for the volume V_{old} and the new state with energy U_{new} for the volume V_{new} according to

$$\Delta E = p_0 (V_{new} - V_{old}) + (U_{new} - U_{old}) + N k_B T \ln \frac{V_{old}}{V_{new}}. \quad (5.17)$$

5.8.2 Caloric properties

The residual enthalpy H^{res} is directly linked to the residual internal energy, pressure and volume of the system

$$H^{\text{res}} = U^{\text{res}} + pV - N k_B T, \quad (5.18)$$

where U^{res} is the residual potential energy. It is defined as the sum of all pairwise interaction energies u_{ij} and the appropriate long range correction ΔU^L , cf. section 5.5

$$U^{\text{res}} = \sum_{i=1}^{N-1} \sum_{\substack{j=i+1 \\ r_{ij} < r_c}}^N u_{ij} + \Delta U^L. \quad (5.19)$$

Massieu potential derivatives *ms2* version 2.0 features evaluating free energy derivatives in a systematic manner, thus greatly extending the thermodynamic property types that can be sampled in single simulation runs. The approach is based on the fact that the fundamental equation of state contains the complete thermodynamic information about a system, which can be expressed in terms of various thermodynamic potentials **MunsterBook** e.g. internal energy $E(N, V, S)$, enthalpy $H(N, p, S)$, Helmholtz free energy $F(N, V, T)$ or Gibbs free energy $G(N, p, T)$, with number of particles N , volume V , pressure p , temperature T and entropy S . These representations are equivalent in the sense that any other thermodynamic property is essentially a combination of derivatives of the chosen form with respect to its independent variables. The form $F/T(N, V, 1/T)$, known as the Massieu potential, is preferred in molecular simulations due to practical reasons **Lus11; Lus12** The statistical mechanical formalism of Lustig allows for the simultaneous sampling of any A_{mn}^r in a single NVT ensemble simulation for a given state point **Lus11; Lus12; Lus94a; Lus94b**

$$\frac{\partial^{m+n}(F/(RT))}{\partial \beta^m \partial \rho^n} \beta^m \rho^n \equiv A_{mn} = A_{mn}^i + A_{mn}^r, \quad (5.20)$$

where R is the gas constant, $\beta \equiv 1/T$ and $\rho \equiv N/V$. A_{mn} can be separated into an ideal part A_{mn}^i and a residual part A_{mn}^r **RowBook** The calculation of the residual part is the target of molecular simulation and the derivatives A_{10}^r , A_{01}^r , A_{20}^r , A_{11}^r , A_{02}^r , A_{30}^r , A_{21}^r and A_{12}^r were implemented in *ms2* for NVT ensemble simulations. The ideal part can be obtained by independent methods, e.g. from spectroscopic data or ab initio calculations. However, it can be shown that for any $A_{mn} = A_{mn}^i + A_{mn}^r$, where $n > 0$, the ideal part is either zero or depends exclusively on the density, thus it is known by default **Lus12** Note that the calculation of A_{00}^r still requires additional concepts such as thermodynamic integration or particle insertion methods. From the first five derivatives A_{10} , A_{01} , A_{20} , A_{11} , A_{02} every measurable thermodynamic property can be expressed (see the supplementary material for a list of properties) with the exception of phase equilibria. A detailed description of the implementation is in the supplementary material, here, only an overview is provided.

The calculation of the derivatives up to the order of $n = 2$ requires the explicit mathematical expression of $\partial U / \partial V$ and $\partial^2 U / \partial V^2$ with respect to the applied molecular interaction pair potential and has to be determined analytically beforehand **Lus11; Lus12** The general formula for $\partial^n U / \partial V^n$ can be found in Ref. **Lus94b** For common molecular interaction pair potentials, like the Lennard-Jones potential **allen1987; frenkel2003** describing repulsive and dispersive interactions, or Coulomb's law, describing electrostatic interactions between point charges, the analytical formulas for $\partial U / \partial V$ and $\partial^2 U / \partial V^2$ can be obtained straightforwardly.

As molecular simulation is currently limited to operate with considerably fewer particles than real systems, the effect of the small system size thus has to be counter-balanced with a contribution to U and $\partial^n U / \partial V^n$ called long range correction (LRC) **allen1987; frenkel2003** The mathematical form of the LRC depends on the molecular interaction potential and the cut-off method (site-site or center-of-mass cut-off mode) applied. For the Lennard-Jones potential, the LRC scheme was well described in the literature for both the site-site **Lus11; Mei06** and the center of mass cut-off mode **Lus94b; Lus88** The reaction field method **RFmethod** was the default choice in the preceding version of *ms2* for the LRC of electrostatic interactions modelled by considering charge distributions on molecules. The usual implementation of the reaction field method combines the explicit and the LRC part in a single pair potential **RFmethod; siteRFmethod** from which

$\partial^n U / \partial V^n$ (including the LRC contribution) is directly obtainable. However, practical applications show that the electrostatic LRC of $\partial U / \partial V$ and $\partial^2 U / \partial V^2$ can be neglected in case of systems for which the reaction field method is an appropriate choice. E.g., the contribution of the electrostatic LRC for a liquid system ($T = 298$ K and $\rho = 45.86$ mol/l) consisting of only 200 water and 50 methanol molecules with a very short cut-off radius of 20% of the edge length of the simulation volume is still $\ll 1\%$ for both $\partial U / \partial V$ and $\partial^2 U / \partial V^2$. The supplementary material contains detailed elaborations on the LRC for the Lennard-Jones potential.

5.8.3 second derivatives

The accessible second derivatives vary with the employed ensemble. In the NVT ensemble, the residual isochoric heat capacity c_v^{res} is determined by fluctuations of the residual potential energy U^{res}

$$c_v^{\text{res}} = \frac{1}{N} \left(\frac{\partial U^{\text{res}}}{\partial T} \right)_v = \frac{1}{k_B (NT)^2} \left(\langle (U^{\text{res}})^2 \rangle - \langle U^{\text{res}} \rangle^2 \right). \quad (5.21)$$

The partial derivative of the potential energy with respect to the volume at constant temperature $(\partial U^{\text{res}} / \partial V)_T$ is determined by fluctuations of the residual potential energy U^{res} and the virial W

$$\left(\frac{\partial U^{\text{res}}}{\partial V} \right)_T = \frac{N}{3V} \left(\frac{1}{k_B T} \left(\langle W \rangle \langle U^{\text{res}} \rangle - \frac{1}{N} \langle W U^{\text{res}} \rangle \right) + \langle W \rangle \right). \quad (5.22)$$

In the NpT ensemble, the second derivatives of the Gibbs energy, namely the residual isobaric heat capacity c_p^{res} , the isothermal compressibility β_T and the volume expansivity α_p are functions of ensemble fluctuations **allen**. The residual isobaric heat capacity c_p^{res} is related to fluctuations of the residual enthalpy H^{res}

$$c_p^{\text{res}} = \frac{1}{N} \left(\frac{\partial H^{\text{res}}}{\partial T} \right)_p = \frac{1}{k_B (NT)^2} \left(\langle (H^{\text{res}})^2 \rangle - \langle H^{\text{res}} \rangle^2 \right). \quad (5.23)$$

To obtain the total isobaric heat capacity c_p , the solely temperature dependent ideal gas contribution $c_p^{\text{id}}(T)$ has to be added. This ideal property is accessible e.g. by quantum chemical methods and can often be found in data bases **rowley**

An analogous relationship links the isothermal compressibility β_T to volume fluctuations in the NpT ensemble

$$\beta_T = -\frac{1}{V} \left(\frac{\partial V}{\partial p} \right)_T = \frac{1}{k_B T \langle V \rangle} \left(\langle V^2 \rangle - \langle V \rangle^2 \right). \quad (5.24)$$

The partial derivative of the residual enthalpy with respect to the pressure at constant temperature $(\partial H^{\text{res}} / \partial p)_T$ is linked to fluctuations of volume and residual internal energy

$$\left(\frac{\partial H^{\text{res}}}{\partial p} \right)_T = V - \frac{N}{T} \left(\langle U^{\text{res}} V \rangle - \langle U^{\text{res}} \rangle \langle V \rangle + p \left(\langle V^2 \rangle - \langle V \rangle^2 \right) \right), \quad (5.25)$$

and the volume expansivity α_p again to volume and residual enthalpy fluctuations

$$\alpha_p = \frac{1}{V} \left(\frac{\partial V}{\partial T} \right)_p = \frac{N}{T^2 \langle V \rangle} \left(\langle H^{\text{res}} V \rangle - \langle H^{\text{res}} \rangle \langle V \rangle \right). \quad (5.26)$$

Note that Eqs. (5.21) to (5.26) are valid for mixtures as well.

The speed of sound c is the velocity of a sound wave traveling through an elastic medium. It is defined by the isothermal compressibility β_T , the volume expansivity α_p , the isobaric heat capacity c_p and the temperature T by

$$c = \left(\frac{1}{M (\beta_T \rho - T \alpha_p^2 / c_p)} \right)^{0.5}, \quad (5.27)$$

where M is the molar mass. In *ms2*, the speed of sound is calculated both for pure components and mixtures in the NpT ensemble.

5.8.4 Transport properties

In *ms2*, transport properties are calculated using equilibrium MD. Here, the fluctuations of a system around its equilibrium state are evaluated as a function of time. The Green-Kubo formalism relates these microscopic fluctuations to the respective transport properties.

Diffusion coefficients. The self-diffusion coefficient D_i is related to the mass flux of single molecules within a fluid. Therefore, the relevant Green-Kubo expression is based on the individual molecule velocity autocorrelation function **gubbins1**

$$D_i = \frac{1}{3N_i} \int_0^\infty dt \langle \mathbf{v}_k(t) \cdot \mathbf{v}_k(0) \rangle, \quad (5.28)$$

where $\mathbf{v}_k(t)$ is the center of mass velocity vector of molecule k at some time t . Eq. (5.28) is an average over all N_i molecules of component i in the ensemble, since all contribute to the self-diffusion coefficient. In binary mixtures, the Maxwell-Stefan diffusion coefficient $\mathbf{D}_{ij}$ is defined by **krishna**

$$\mathbf{D}_{ij} = \frac{x_j}{x_i} \Lambda_{ii} + \frac{x_i}{x_j} \Lambda_{jj} - \Lambda_{ij} - \Lambda_{ji}, \quad (5.29)$$

where $x_i = N_i/N$ and Λ_{ij} can be written in terms of the center of mass velocity

$$\Lambda_{ij} = \frac{1}{3N} \int_0^\infty dt \left\langle \sum_{k=1}^{N_i} \mathbf{v}_{i,k}(0) \cdot \sum_{l=1}^{N_j} \mathbf{v}_{j,l}(t) \right\rangle. \quad (5.30)$$

From the expressions above, the collective character of the Maxwell-Stefan diffusion coefficient is evident. This leads to significantly less data for a given system size and time step and therefore to larger statistical uncertainties than in case of the self-diffusion coefficient. Note that equations for the Maxwell-Stefan diffusion coefficient are implemented in *ms2* both for binary and ternary mixtures **krishna**

Shear viscosity. The shear viscosity η , as defined by Newton's "law of viscosity, is a measure of the resistance of a fluid to a shearing force **hoheisel** It is associated with the momentum transport under the influence of velocity gradients. Hence, the shear viscosity can be related to the time autocorrelation function of the off-diagonal elements of the stress tensor $\mathbf{J}_p$ **gubbins1**

$$\eta = \frac{1}{Vk_B T} \int_0^\infty dt \langle J_p^{xy}(t) \cdot J_p^{xy}(0) \rangle. \quad (5.31)$$

Averaging over all three independent elements of the stress tensor, i.e. J_p^{xy} , J_p^{xz} and J_p^{yz} , improves the statistics. The component J_p^{xy} of the microscopic stress tensor $\mathbf{J}_p$ is given in terms of the molecular positions and velocities by **hoheisel**

$$J_p^{xy} = \sum_{i=1}^{N-1} m v_i^x v_i^y - \sum_{i=1}^{N-1} \sum_{j=i+1}^N \sum_{k=1}^n \sum_{l=1}^n r_{ij}^x \frac{\partial u_{ij}}{\partial r_{kl}^y}. \quad (5.32)$$

Here, the lower indices l and k indicate the n interaction sites of a molecule and the upper indices x and y denote the spatial vector components, e.g. for velocity v_i^x or site-site distance r_{ij}^x . The first term of Eq. (5.32) is the kinetic energy contribution and the second term is the potential energy contribution to the shear viscosity. Consequently, the Green-Kubo integral (5.31) can be decomposed into three parts, i.e. the solely kinetic energy contribution, the solely potential energy contribution and the mixed kinetic-potential energy contribution **hoheisel**

Bulk viscosity. The bulk viscosity η_v refers to the resistance to dilatation of an infinitesimal volume element at constant shape **barton** The bulk viscosity can be calculated by integration of the time-autocorrelation function of the diagonal elements of the stress tensor and an additional term that involves the product of pressure p and volume V , which does not appear in the shear viscosity, cf. Eq. (5.31). In the NVE ensemble, the bulk viscosity is given by **gubbins1**; **steele**

$$\eta_v = \frac{1}{Vk_B T} \int_0^\infty dt \langle (J_p^{xx}(t) - pV(t)) \cdot (J_p^{xx}(0) - pV(0)) \rangle. \quad (5.33)$$

The diagonal component J_p^{xx} of the microscopic stress tensor $\mathbf{J}_p$ is defined analogous to Eq. (5.32). The statistics of the ensemble average in Eq. (5.33) can be improved using all three independent diagonal elements of the stress tensor J_p^{xx} , J_p^{yy} and J_p^{zz} . Eqs. (5.32) and (5.33) can directly be applied for mixtures.

electric conductivity The evaluation of the electric conductivity σ was implemented in *ms2* version 2.0, being a measure for the flow of ions in solution. The Green-Kubo formalism **gubbins1972** offers a direct relationship between σ and the time-autocorrelation function of the electric current flux $\mathbf{j}_e(t)$ **HansenBook**

$$\sigma = \frac{1}{3Vk_B T} \int_0^\infty \langle \mathbf{j}_e(t) \cdot \mathbf{j}_e(0) \rangle dt, \quad (5.34)$$

where k_B is Boltzmann's constant. The electric current flux is defined by the charge q_k of ion k and its velocity vector $\mathbf{v}_k$ according to

$$\mathbf{j}_e(t) = \sum_{k=1}^{N_j} q_k \cdot \mathbf{v}_k(t), \quad (5.35)$$

where N_j is the number of molecules of component j in solution. Note that only the ions in the solution have to be considered, not the electro-neutral molecules. For better statistics, σ is sampled over all independent spatial elements of $\mathbf{j}_e(t)$.

thermal conductivity In the previous version of *ms2* the determination of the thermal conductivity by means of the Green-Kubo formalism was implemented for pure substances only. In the present release, the calculation of the thermal conductivity was extended to multi-component mixtures. The thermal conductivity λ is given by the autocorrelation function of the elements of the microscopic heat flow J_q^x

$$\lambda = \frac{1}{Vk_B T^2} \int_0^\infty dt \langle J_q^x(t) \cdot J_q^x(0) \rangle. \quad (5.36)$$

In mixtures, energy transport and diffusion occur in a coupled manner, thus, the heat flow for a mixture of n components is given by **evans1**

$$\begin{aligned} \mathbf{J}_q = & \frac{1}{2} \sum_{i=1}^n \sum_{k=1}^{N_i} \left[m_i^k (v_i^k)^2 + \mathbf{w}_i^k \mathbf{I}_i^k \mathbf{w}_i^k + \sum_{j=1}^n \sum_{l \neq k}^{N_j} u(r_{ij}^{kl}) \right] \cdot \mathbf{v}_i^k \\ & - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \sum_{k=1}^{N_i} \sum_{l \neq k}^{N_j} \mathbf{r}_{ij}^{kl} \cdot (\mathbf{v}_i^k \cdot \frac{\partial u(r_{ij}^{kl})}{\partial \mathbf{r}_{ij}^{kl}} + \mathbf{w}_i^k \mathbf{I}_{ij}^{kl}) - \sum_{i=1}^n h_i \sum_{k=1}^{N_i} \mathbf{v}_i^k, \end{aligned} \quad (5.37)$$

where $\mathbf{w}_i^k$ is the angular velocity vector of molecule k of component i and $\mathbf{I}_i^k$ its matrix of angular momentum of inertia. $u(r_{ij}^{kl})$ is the intermolecular potential energy and $\mathbf{I}_{ij}^{kl}$ is the torque due to the interaction of molecules k and l . The indices i and j denote the components of the mixture. h_i is the partial molar enthalpy. It has to be specified as an input in the *ms2* parameter file and can be calculated from *NpT* simulations.

5.8.5 configurational properties

The **radial distribution** function (RDF) $g(r)$ is a measure for the microscopic structure of matter. It is defined by the local number density around a given position within a molecule $\rho^L(r)$ in relation to the overall number density $\rho = N/V$

$$g(r) = \frac{\rho^L(r)}{\rho} = \frac{1}{\rho} \frac{dN(r)}{dV} = \frac{1}{4\pi r^2 \rho} \frac{dN(r)}{dr}. \quad (5.38)$$

Therein, $dN(r)$ is the differential number of molecules in a spherical shell volume element dV , which has the width dr and is located at the distance r from the regarded position. $g(r)$ can be evaluated for every molecule of a given species.

In the present release of *ms2*, the RDF can be calculated during MD simulation runs for pure components and mixtures on the fly. The RDF is sampled between all LJ sites. In order to evaluate RDFs for arbitrary positions, say point charge sites, superimposed dummy LJ sites with the parameters $\sigma = \epsilon = 0$ have to be introduced in the potential model file by the user.

The **residence time** τ_j defines the average time span that a molecule of component j remains within a given distance r_{ij} around a specific molecule i . It is given by the autocorrelation function

$$\tau_j = \int_{t=0}^{\infty} \left\langle \frac{1}{n_{ij}(0)} \sum_{k=1}^{n_{ij}(0)} \Theta_k(t) \Theta_k(0) \right\rangle dt, \quad (5.39)$$

where t is the time, $n_{ij}(0)$ the solvation number around molecule i at $t = 0$ and Θ is the Heaviside function, which yields unity, if the two molecules are within the given distance, and zero otherwise. Following the proposal of Impey et al. **Impey1983** the residence time explicitly allows for short time periods during which the distance between the two molecules exceeds r_{ij} . Also, the solvation number n_{ij} can be evaluated on the fly

$$n_{ij} = 4\pi\rho_j \int_0^{r_{\min}} r^2 g_{ij}(r) dr, \quad (5.40)$$

where ρ_j is the number density of component j and $r_{\min}$ is the distance up to which the solvation number is calculated.

5.8.6 Henry' s law constant

The solubility of solutes in solvents is characterized by the Henry's law constant. In solvents of a given composition, it is solely a function of temperature. The Henry's law constant H_i is related to the residual chemical potential of the solute i at infinite dilution $\mu_i^{res,\infty}$ as defined in e.g. **shing** by

$$H_i = \rho k_B T \exp \left(\frac{\mu_i^{res,\infty}}{k_B T} \right), \quad (5.41)$$

where ρ denotes the solvent density. The residual chemical potential of a component at infinite dilution can often be calculated using Widom's test molecule method. In this case, a simulation of the solvent is performed in the NpT ensemble and the saturated vapor pressure of the solvent, while the solute is introduced as an additional component of the mixture with a molar fraction of zero. Then, the solute is only added in form of test molecules to determine its chemical potential at infinite dilution. For dense phases, the chemical potential and thus the Henry's law constant can be calculated with the gradual insertion method, if Widom's method fails. The number of solute molecules in the simulation is reduced to one, representing the infinitely diluted molecule.

5.8.7 second virial coefficient

The second virial coefficient is related to the intermolecular potential by **gray**

$$B = -2\pi \int_0^\infty dr_{ij} \left\langle \exp \left(-\frac{u_{ij}(r_{ij}, \omega_i, \omega_j)}{k_B T} \right) - 1 \right\rangle_{\omega_i, \omega_j} r_{ij}^2, \quad (5.42)$$

where the $\langle \dots \rangle$ brackets indicate the average over the orientations ω_i and ω_j of two molecules i and j separated by a center of mass distance r_{ij} . The integrand, called Mayer's f -function, is evaluated at numerous distances in an appropriate range. *ms2* evaluates Mayer's f -function by averaging over numerous random orientations at each radius.

5.9 statistical Errors

5.10 special features

5.10.1 Vapour-liquid equilibrium

5.10.2 osmotic pressure

5.10.3 Hydrogen bonding

5.10.4 humid air Ensemble

6 Potential Models in the ms2-distribution

7 Computational Aspects

7.1 Sequential Version

7.2 Parallelization

The source code is implemented for an efficient parallelism with distributed memory, using the MPI **Forum2009** standard for communication. The program uses the following MPI calls:

- MPI_Init, MPI_Finalize, MPI_Comm_rank, MPI_Comm_size to set up basic MPI functionality
- MPI_Abort to stop the program in case of an error
- MPI_Barrier, MPI_Wtime and MPI_Wtick within the stopwatch class
- MPI_Bcast, MPI_Reduce, MPI_Allreduce for simulation data exchange

Exclusively collective communication is used, an approach suitable for molecular simulations, where the cut-off radius is in the order of the simulation volume edge length. For MD simulations with *ms2*, only the force calculation is parallelized using the molecule decomposition according to Plimpton **plimpton1993**. The interaction matrix is rearranged such that the number of interaction partners is almost equally distributed in the matrix. This is achieved by calculating the interactions between molecule 1 and N as interactions between molecules N and 1, cf. Figure 7.1. The parallelization scheme then distributes the calculation among N_P processors with an almost equal work load such that every processor executes N/N_P rows of the interaction matrix. The master process reduces then all the force components to sum up the resulting molecular forces on each molecule.

The MC code is parallelized exploiting the stochastic nature of the simulation method. Starting from an equilibrated state in the simulation volume, the molecular configuration is copied multiple times into different volumes. Then, each copy runs independently in parallel, but with a different random number seed, to calculate the thermodynamic properties. At the end, the data from all copies are gathered and averaged.

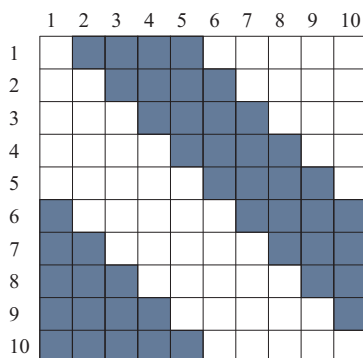


Abbildung 7.1: Schematic of the molecule decomposition parallelization by Plimpton **plimpton1993**

7.3 Vectorization & Memory

The program *ms2* specifically accounts for vector-parallel machines. All the main information needed in the computationally costly inner loops of the calculation, like the positions of the molecules and their sites, are stored in vectors that are accessed sequentially. This allows for an efficient use of memory on vector-parallel machines.

For the evaluation of one configuration in *ms2*, e.g. the position vectors are loaded once and then distributed among all processors. Then, additional information like interaction partners of each individual molecule is computed, tabulated in vectors and communicated. The order of this data follows the order of all other vectors, e.g. the position vector. An example of a storage vector is shown in Table 7.1. The appropriate order allows for a serial access to the data in all vectors in the random access memory. Therefore, the calculations can be vectorized, increasing the efficiency.

Tabelle 7.1: Listing of the interaction partners for a molecule i , indicating the assembly in the storage vector.

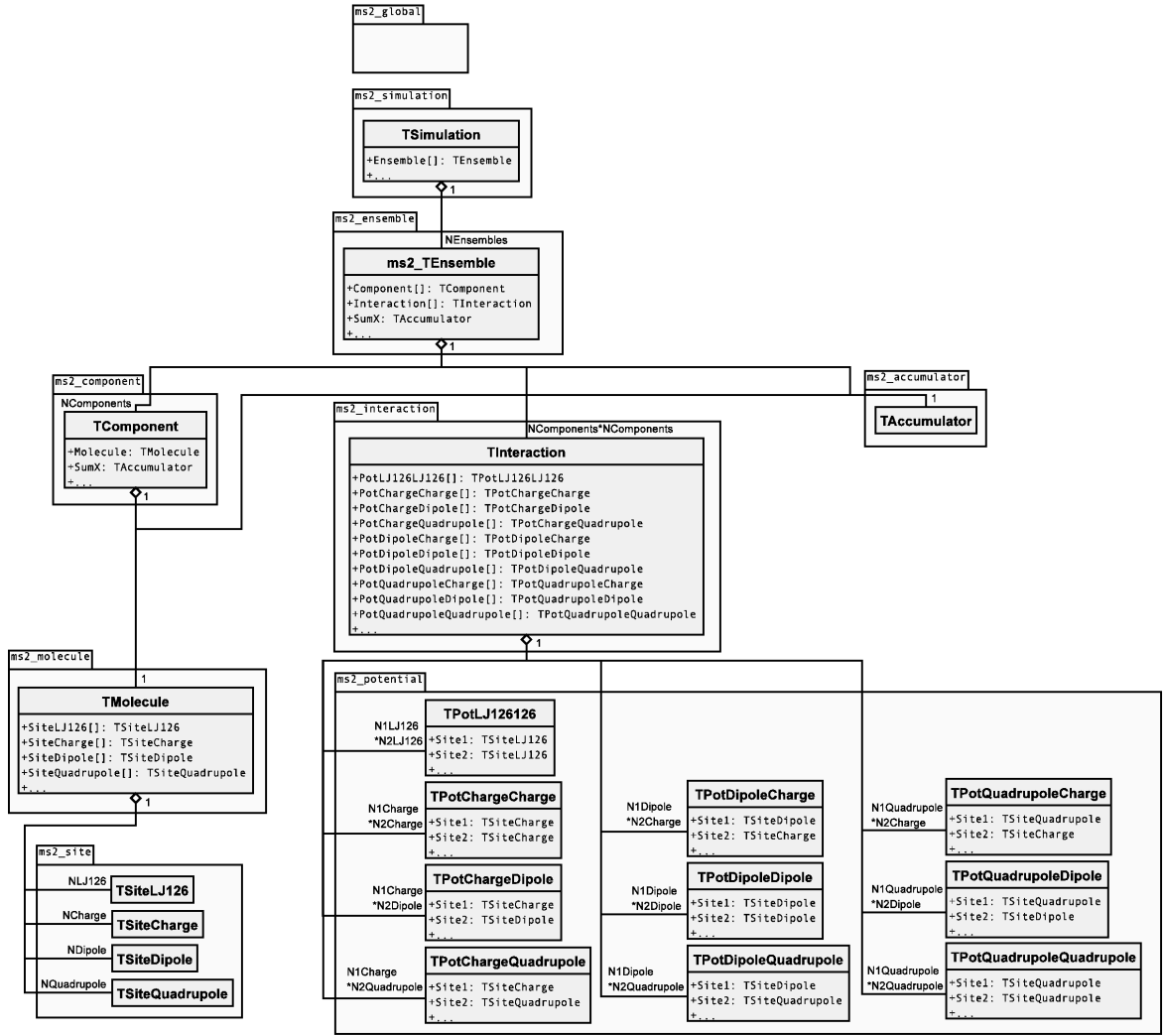
Molecule i	Interaction list for molecule i within the storage vector				
1	2	4	6	...	10
3		4	7	...	10
4			8	...	10

7.4 Benchmarks/ Test case

7.5 Structure of program code

This section discusses the code design and implementation issues of *ms2* in detail. These explanations of the source code are intended to help understanding the program and to encourage the reader to further develop and improve the code. New developers should get a smooth entry and the possibility to make fast adaptations to new problems. Due to the experience of the core developers and the suitability for an efficient numerical code, Fortran90 was chosen as programming language. The program makes extensive use of the concepts introduced with Fortran90, notably modular programming. A wide variety of different simulation setups is possible, each requiring different computations. The most significant difference between the setups lies with the choice of MD or MC. While MD features extensive force calculations in order to update the molecular positions, MC is based on a Markov chain, where a molecular move is accepted with a certain probability. However, there are many other choices regarding the setup. This variety leads to a strong need for a highly modular structure, where modules have clear-cut interfaces. Given such a modularity, the appropriate functionalities can be combined to the desired simulation, while leaving out any unnecessary code¹. Besides avoiding if-statements at computationally demanding points and therefore leading to a better data flow, this allows for an efficient implementation of new functionalities, as existing modules can be used whenever feasible. In *ms2*, a stringent philosophy regarding modularization was followed, which is discussed in section ???. The runs need to be fast and scale well with increasing number of processors in order to achieve short response times. *ms2* is parallelized with the Message Passing Interface (MPI) **Forum2009** While MD needs synchronization after every time step due to its deterministic nature, MC can be run

¹Unnecessary for the current setup.

Abbildung 7.2: UML class diagram of *ms2*, showing object attributes.

embarrassingly parallel, only requiring synchronization at the very end of a run². Furthermore, the by far most expensive part of the code, calculating forces and potential energies, is highly vectorized and optimized.

In order to keep the class hierarchy flat, *ms2* does not make use of inheritance and hence polymorphism. E.g. the module *ms2_site* contains the classes *TSiteLJ126*, *TSiteCharge*, *TSiteDipole* and *TSiteQuadrupole*, which have data and functionalities in common, but they are not derived from a common class. The class structure and the relations between the classes is organized in six levels, as depicted in Figure 7.2. Every class is located on a certain level and hence has data and modules relevant to its level. The six levels are intuitive:

1. Global – all data and functions of global use are handled. Scope: whole code
2. Simulation – setup and control of the simulation. Scope: simulation framework
3. Ensemble – the required ensemble is set up and initialized. Scope: molecular ensemble

²Therefore, parallelization can be achieved by simply executing multiple MC runs simultaneously and averaging over all runs, cf. section ??.

4. Component – data and functions dealing with all molecules of a given molecule type. Scope: one molecule species
5. Molecule – data regarding the basic structure of a molecule. Scope: one molecule
6. Site – position of sites with respect to a molecule. Scope: site of one molecule

First level - global The first level contains subroutines and variables that are needed globally in all parts of the simulation. The variables are mainly natural constants like the Boltzmann constant k_B , the Avogadro constant N_A , etc. Additionally, variables defining the simulation setup for the given run are stored, e.g. the simulation technique and the ensemble type. These variables determine the type of calculation and are assigned according to user specifications on subsequent levels in *ms2*. Subroutines that are stored on this level are also of global use in *ms2*. These are basic routines for input into the simulation tool and output into files, as well as more advanced routines for automated communication between hardware and program via signals. The latter routines are of particular interest for running *ms2* on high performance clusters, where possible data loss can be avoided by automated communication between cluster nodes and the program.

Second level - simulation On the second level, the simulation flow is controlled. The simulation is initiated, started and sustained. This includes controlling the length of the simulation and its output, which is further discussed in section ???. User specifications concerning the simulation setup are read and the corresponding global variables are assigned accordingly. Furthermore, all simulation quantities and averages are analyzed and written to file. **Third level - ensemble** On the third level, the simulation is organized according to the desired ensemble specified by the user. The respective ensemble is initialized, by characterizing the ensemble setup and defining e.g. the volume and the composition of the molecular species in the simulation volume. In addition, the initial positions and velocities of all molecules are distributed. Furthermore, the routines for the subsequent simulation are assigned on this level. The time evolution or the Markov chain, respectively, is performed and the ensemble averages of thermodynamic properties are calculated during the simulation run. **Fourth level - component** On this level, the molecule species (component) and their interactions are addressed. The class structure has three distinct branches. While the first branch deals with issues regarding structure and properties of one component at a time, the second branch deals with interactions between molecules of one or two components. The third branch contains all routines for the accumulation of data, determining the average value of the data as well as their statistical uncertainties. The first two branches will be discussed in more detail.

The first branch deals with the structure of one individual component. An instance of the class component stores center of mass position, orientation, velocity etc. of every molecule of one species, e.g. H_2O . The contained modules deal with calculations all molecules of this species, e.g. modules for solving Newton's equations of motion, kinetic energy calculation, atom to molecule transformation and vice versa. Furthermore, this branch is responsible for the correct initialization. It reads the molecular model for every component, calculates positions, introduces the reduced quantities, etc.

The second branch deals with interactions between molecules of one or two components. Therefore, this branch contains all force and potential energy calculations, which are the computationally most expensive parts by far. This branch is highly vectorized to achieve a fast calculation, in particular if executed in vector-parallel. The modules in this branch are simulation technique specific, i.e. MD or MC. In a MD simulation, first, the interaction partners of every molecule are determined, i.e. other molecules or sites within the cut-off radius. Then the interaction energies and virial contributions as well as the resulting forces and torques are calculated and summed up for every molecule.

In a MC simulation, the modules are designed to calculate only the energy and the virial. Forces are not calculated and the corresponding algorithms are therefore entirely omitted in these modules. For MD, all interactions of one component-component pair³ are calculated in one call of the module

³Please note that a component-component pair can consist of one component paired with itself.

and they are therefore called once for every component-component pairing. For MC, the modules are finer-grained, calculating all interactions of one component-component pair by determining every molecule-molecule interaction individually. The corresponding modules are thus called more frequently than in case of MD. **Fifth level - molecule** The basic information on molecules is stored on this level, such as mass, moment of inertia tensor, rotational degrees of freedom, etc. Higher level modules access this data via the class molecule.

Sixth level - site Here, the assembly of a molecule is stored. The data is used by higher level modules to calculate the site positions from the molecular positions and orientations. This allows for the calculation of site-site interactions. It should be noted that *ms2* only integrates (MD) or accepts (MC) center of mass positions and orientations. Site specific data is calculated for every simulation step on the fly.

Example The modular structure is discussed by highlighting the calculation process of a MD simulation of a pure fluid modeled by two LJ sites and two point charges in the *NVT* ensemble. During the initialization process, the modules for memory allocation and definition of the simulation scenario are executed. This process is not unique for one of the hierarchical levels, but takes places on all levels, cf. Figure 7.2. On level one, the simulation is delimited to "molecular dynamics", therefore all modules concerning MC simulation are neglected entirely. On level two, the "initialization" modules define the simulation environment by setting ensemble specific properties like simulation volume, temperature as well as the thermostat and integration scheme, etc. While the use of the first modules mentioned here is required in each simulation, independently of MD or MC, e.g. the trajectory calculation is specific for MD. However, the modular structure helps keeping the efficiency of the program by initiating only the simulation-specific molecule propagation. The same effect of modules can be found in the initiation process on lower levels. Here, the exact composition of the molecular species in the simulation volume (module component), the description of the molecule (module molecule) and the storage of the positions, velocities and forces of all LJ sites and charges (module sites) is determined. All other modules, like treating additional components for mixtures or containing characteristic properties of dipoles or quadrupoles, are fully omitted by *ms2*, since these are not used throughout this simulation. Once the simulation has been initialized, a set of modules is invoked, which deals with maintaining the molecular simulation. On level one, this includes modules of global use for any simulation, e.g. limiting the execution time, etc. On level two, Newton's equations of motion are numerically integrated, requiring information about the fluid's composition, the positions and orientations of the molecules etc. provided by modules of the respective level. The thermostat is applied and the calculation of thermodynamic properties is performed. The forces, torques and energies are calculated in the second branch on the level component. For a MD simulation, this covers routines for the calculation of energies as well as forces at the same time, whereas in contrast for a MC simulation, the modules only calculate energies and virial contributions. A further set of modules deals with the accumulation of data. These modules are designed to store data and evaluate the statistics of the produced data. The actual thermodynamic properties are calculated on the ensemble level. The last set of modules deals with the output, where the results of the molecular simulation are written to file. These modules are called independent on the user settings in all simulation scenarios.

8 Tools

The tools described in the following are intended to facilitate the handling of *ms2*. They should allow for an easy start of molecular simulations with *ms2* and give access to all output data generated by *ms2*.

8.1 *ms2par*

The GUI-based feature tool *ms2par* allows for a convenient generation of *.par files according to user specifications, cf. Figure ???. It assigns the molecular model, ensemble, thermodynamic state point, time step, number of equilibration steps, etc. Furthermore, the tool proposes a cut-off radius based on the input data. After the user completes the specifications, the program generates the file *.par in ASCII format, to be read by *ms2*. Applying *ms2par* is simple and should be intuitive even for new users. *ms2par* is a java application, thus it runs independently on all operating systems.

8.2 *ms2potmod*

8.3 *ms2chart*

ms2chart is a GUI-based java applet for evaluating the simulation results from *ms2*, cf. Figure ???. This tool plots the evolution of properties calculated by *ms2*. The properties shown on the different graph axes are chosen from a drop down menu and the plot is shown directly in the GUI. For a better analysis, *ms2chart* allows various features: plotting block averages as well as simulation averages into the same plot, changing the design of the plot, individual labeling of the axes and adjusting the frame detail of the plot. All plots can be exported in png format. If needed, the simulation details can be displayed, i.e. the specifications in the *.par file, the results in the *.res file and the summary in the *.log file.

The analysis program *ms2chart* can be executed at any time of the simulation, i.e. also while the simulation is still ongoing. There is no loss of data, if it is run on the fly. The tool *ms2chart* is easy to understand and can be handled intuitively. It allows for an easy first analysis of the *ms2* simulation results. Advanced users will use *ms2chart* for a quick check of their simulation results, such as whether the equilibration time was appropriate or for a fast analysis, which is important if extensive series of simulation runs are performed.

8.4 *ms2molecules*

The program *ms2molecules* is a visualization tool for *ms2*. The program displays the molecular trajectories that are stored by *ms2* as a series of configurations in the *.vim file. *ms2molecules* visualizes molecular sites by colored spheres. The colors are user defined. The size of a sphere is by default proportional to the LJ size parameter σ , but can be changed manually in the first lines of the *.vim-file. This feature facilitates monitoring a component of interest by reducing the size of other components, e.g. solvents. Other features like zooming into or out of the simulation

volume as well as rotating the simulation volume also facilitate the analysis of the simulation. The visualization can be exported via snapshots in the jpg format.

The program is based on OpenGL and written in C. The handling of the program is simple and console based. Requirements for the feature tool are OpenGL in a Windows or Linux environment. Figure ?? shows a snapshot of a ternary mixture, taken with the program *ms2molecules*, which is convenient to gain insight into the trajectory of the system, or the state of the system, respectively.

8.5 res2gepar

9 Examples & Tutorials

10 Error Messages

Symbolverzeichnis

Formelzeichen

Symbol	Bedeutung	Einheit
a	Länge	m
b	Breite	m
c	spezifische Wärmekapazität	$\text{kJ}/(\text{kg K})$
ℓ	charakteristische Länge	m
Re	Reynoldszahl	
w	Strömungsgeschwindigkeit	m/s

Griechische Symbole

Symbol	Bedeutung	Einheit
λ	Wärmeleitfähigkeit	$\text{W}/(\text{m K})$
ν	kinematische Viskosität	m^2/s
ϱ	Dichte	kg/m^3

Indizes

Symbol	Bedeutung
L	Luft
p	isobar

Abkürzungsverzeichnis

eps	Encapsulated PostScript
pdf	Portable Document Format
svg	Scalable Vector Format